

Performance dell'Hardware

Quando progettiamo o eseguiamo un'applicazione parallela, il nostro obiettivo primario è massimizzarne l'efficienza. In generale, la performance complessiva di un sistema di calcolo è determinata da tre fattori cruciali: il tempo impiegato per ottenere un risultato, la quantità di risorse informatiche utilizzate e l'energia consumata. Le prestazioni non dipendono solo dalla "forza bruta" dell'hardware (CPU, GPU, velocità della memoria, capacità della rete e dello storage). Esse sono fortemente influenzate dalla qualità del software, che include l'efficienza degli algoritmi scelti, l'organizzazione dei dati in memoria, la capacità del codice di sfruttare appieno l'hardware a disposizione (*hardware exploitation*) e l'ottimizzazione garantita dal sistema operativo e dai compilatori.

Metriche di misurazione: peak vs sustained performance

Per valutare un sistema, utilizziamo due concetti fondamentali:

- **Theoretical Peak Performance:** è una stima puramente matematica e teorica delle prestazioni massime che un componente hardware può raggiungere, basata esclusivamente sulle sue specifiche tecnologiche (frequenza di clock, numero di core, ecc.).
- **Sustained Performance (Throughput):** rappresenta le prestazioni *effettive* e misurate sul campo. Si ottiene facendo eseguire alla macchina dei programmi standardizzati chiamati **Benchmark**.

Nota sulla classifica TOP500: Nella famosa lista dei supercomputer più potenti al mondo, vengono riportati entrambi i valori. L'indice **Rpeak** indica la Theoretical Peak Performance, mentre l'indice **Rmax** indica la Sustained Performance ottenuta eseguendo lo specifico benchmark HPL (High Performance Linpack).

Analisi della CPU: dai MIPS ai GFLOPS

In passato, la potenza di una CPU si misurava in MIPS (milioni di istruzioni eseguite per secondo). Oggi, in ambito HPC, si valutano quasi esclusivamente le operazioni matematiche complesse misurandole in **FLOPS**, utilizzando multipli come MFLOPS, GFLOPS o TFLOPS. Le prestazioni massime teoriche di un singolo core si calcolano con una formula semplice $FLOPS = Clock \times IPC$.

Dove:

- **Clock:** è la frequenza di lavoro del processore (es. cicli al secondo).
- **IPC (Instructions Per Cycle):** è il numero di operazioni in virgola mobile che il core riesce a completare in un singolo ciclo di clock.

Un fattore critico che influenza l'IPC è la **precisione** del dato elaborato. I calcoli possono essere eseguiti in singola precisione (FP32, a 32 bit), doppia precisione (FP64, a 64 bit) o mezza precisione (FP16, a 16 bit). Più i dati sono "larghi" (come in FP64), meno operazioni il processore riuscirà a far stare in un singolo ciclo, dimezzando di fatto i FLOPS teorici rispetto a precisioni inferiori.

SIMD e FMA: il vero motore della CPU

Un processore moderno, grazie all'architettura a pipeline, ha un IPC base pari a 1 (un risultato per ciclo di clock). Tuttavia, le vere prestazioni esplodono grazie a due tecnologie:

1. **Istruzioni Vettoriali (SIMD):** produttori come Intel e AMD hanno inserito registri dedicati molto ampi. Con l'estensione AVX-512 (registri a 512 bit), un core può processare simultaneamente in un solo ciclo di clock ben 16 operazioni in singola precisione o 8 in doppia precisione (IPC=8).

2. **FMA (Fused Multiply-Add):** Poiché molti algoritmi (come le moltiplicazioni tra matrici) richiedono di moltiplicare due numeri e sommarli a un terzo ($a \times b + c$), l'istruzione FMA esegue questa doppia operazione in un unico ciclo di clock, raddoppiando l'efficienza (IPC x 2).

Architettura e Performance delle GPU

Le GPU (Graphics Processing Unit) affrontano il parallelismo in modo diverso. Qui, l'unità elementare di calcolo è il **Thread**, che viene eseguito su un piccolo core dedicato chiamato *Streaming Processor (SP)* o *CUDA core*. Per massimizzare l'efficienza, i thread non vengono gestiti singolarmente, ma raggruppati in blocchi di 32 chiamati **Warp**.

Questi Warp sono amministrati da uno **Streaming Multiprocessor (SM)**. Un singolo SM è un concentrato di potenza che include decine di SP (es. 64 nelle GPU P100 e A100), uno scheduler per i Warp, una memoria velocissima privata per i registri di ogni thread, e una cache L1 (Shared Memory) condivisa tra tutti i thread di quell'SM. L'intera scheda GPU è poi composta da decine di SM (108 nella NVIDIA A100) che si affacciano su una vasta memoria globale (DRAM).

Il calcolo della *Peak Performance* di una GPU segue logiche simili a quelle della CPU:

$$\text{Peak} = \text{Freq Boost} \times 2 (\text{FMA}) \times \text{Numero totale di CUDA Core}$$

Ad esempio, una NVIDIA A100, lavorando in doppia precisione a 1410 MHz con 3456 core, tocca i **9.7 TFLOPS** teorici.

La gerarchia di Memoria e il Modello Roofline

Avere processori potentissimi è inutile se i dati non arrivano in tempo. La memoria è organizzata in una gerarchia piramidale che bilancia velocità, vicinanza al core e capacità:

- **Registers:** Memoria istantanea interna al core (~0.3 ns).
- **Cache L1, L2, L3:** Memorie intermedie molto veloci ma di capienza limitata (da 1 a 12 ns).
- **Main Memory (RAM):** La memoria di sistema, più capiente ma più lenta (~60 ns).
- **Storage (SSD/HDD):** Archiviazione permanente, estremamente lenta (da 0.1 a 3 ms).

Se la memoria non riesce a fornire i dati al ritmo richiesto dall'unità di calcolo, essa diventa un "collo di bottiglia".

Il Modello Roofline

Per capire se il nostro programma sta soffrendo a causa della CPU o della memoria, si utilizza l'analisi *Roofline*.

Questo modello grafico si basa sul concetto di **Arithmetic Intensity** (Intensità Aritmetica), ovvero il rapporto tra le operazioni matematiche (FLOPs) e i dati caricati (Bytes).

- Se un'applicazione fa poche operazioni su tanti dati (bassa intensità, es. una semplice somma tra vettori), il grafico mostrerà che siamo **Memory-bound** (o Bandwidth-bound): la prestazione è frenata dalla larghezza di banda della memoria.
- Se l'applicazione fa moltissimi calcoli sui dati appena caricati (alta intensità, es. simulazioni fisiche o moltiplicazioni di matrici), siamo **Compute-bound**: la memoria fa in tempo a rifornire la CPU, e il limite diventa la Peak Performance assoluta del processore.

Misurare e Analizzare le Performance (Profiling)

Come traduciamo tutta questa teoria in pratica? Utilizzando strumenti software per misurare le prestazioni sostenute:

- **Benchmark:** Software standardizzati per testare la macchina. Per le CPU si utilizza la suite *SPEC CPU2017*. Per calcoli matriciali intensivi in virgola mobile, si usa la libreria *LINPACK* e la sua variante *HPL*, adottata per stilare la classifica TOP500.
- **Temporizzazione nel codice:** Per misurare internamente un algoritmo, in linguaggio C si usa `clock_gettime()` per ottenere il tempo di orologio reale (wall clock) o il tempo CPU netto. Le librerie HPC offrono funzioni native: `omp_get_wtime()` per OpenMP e `MPI_Wtime()` per MPI.
- **Profiling:** È la tecnica che analizza nel dettaglio il comportamento del programma in esecuzione, identificando dove spende più tempo o risorse, senza dover alterare il codice sorgente. In ambiente GNU si usa `gprof` (compilando con il flag `-pg`, che genera un file di report `gmon.out`). Per il mondo accelerato da GPU, NVIDIA offre strumenti analitici proprietari dedicati, come `nvprof`.